

Vest Power Module V1.0

By: Gonalo Ramos

Date: 18/02/2026

Tags: Power Module; Battery Consumption; Mean Current; Max Current Drawn;

Introduç o

Este documento apresenta uma estimativa do consumo real do **Monitoring Vest** e da capacidade de baterias necess rias para 8h de utiliza  o, assim como outras v ri veis, com I_{mean} , I_{max} do dispositivo. Tamb m   apresentado e explicado em detalhe o algoritmo do ficheiro `VestConsumption_UWB_NotAdressed.m`, utilizado para parametrizar as estimativas.

UPDATE

V1.0 - O consumo estimado pelo o UWB est  incorreto, dado que o algoritmo de utiliza  o ainda n o foi determinado. Duty_Cycle do LoRa ainda n o est  bem ajustado.   esperado que o consumo total aumente um pouco com a adi  o futura do UWB. Alguns valores ainda podem ter que vir a ser ajustados.

Table of Contents

- [Vest Power Module V1.0](#)
 - [Introdu o](#)
 - [UPDATE](#)
 - [Table of Contents](#)
 - [Description](#)
 - [Notas](#)
 - [Results](#)
 - [Code Methodology](#)
 - [Sensors Energy Consumption](#)
 - [Bus Energy Consumption](#)
 - [LoRa Energy Consumption](#)
 - [ESP32 Energy Consumption](#)
 - [Battery Capacity](#)
 - [Peak Current](#)

Description

Para calcular a capacidade das baterias, temos que calcular a **corrente m dia** que o sistema precisa e multiplicar pelo **n mero de horas** que o sistema deve ficar ligado.

Para aumentar a dura  o das baterias, temos de otimizar o consumo do coleite. Para isso, foi tido em conta que ir mos tirar o m ximo proveito dos **FIFOs presentes** nos sensores de forma para reduzir overheads de dados nos barramentos. Al m disso, um bom algoritmo de tracking e de comunica  o tem que ser projetado para reduzir o consumo total.

Neste documento foi calculado, além das baterias, o **pico de corrente máxima** que o sistema pode drenar, de forma a ser possível comprar um LDO que suporte esses picos.

Para a realização das estimativas, foram definidos alguns valores base para determinar os valores. Entre estes são:

Freq ESP	Freq I2C	Freq SPI	Baudrate UART	Fs TEMP	Fs PPG	Fs IMU	Fs GNSS
80 MHz	400 KHz	10 MHz	115200	0.1 Hz	100 Hz	100 Hz	0.1 Hz

Notas

O módulo GNSS têm $T_{conversion} = 6 \text{ s}$. Neste caso, o valor refere-se ao tempo que o GNSS precisa para receber os dados do satélite para determinar a sua Lat, Long, etc. e pode variar (num floresta) entre 5 a 15 segundos em média. Se não obter sinal pode ir até infinito. Para tornar minimamente eficiente bloqueiei o valor a 6s (podemos fazer um TTL - time to leave, para quando o GNSS não apanha sinal), no entanto, como é bastante elevado (na ordem de segundos), faz o consumo do GNSS disparar.

Este valor não pode passar de $T_{conversion} < T_{cycle} - T_{wake} - (\text{ESP sleeping time})$. Se quisermos aumentar o tempo que ele pode receber sinal, temos que aumentar o T_{cycle} (por exemplo para 15s).

Results

Utilizando um $T_{cycle} = 10 \text{ s}$ como base, obtemos os seguintes valores:

$I_{mean} = 51.10 \text{ mA}$;

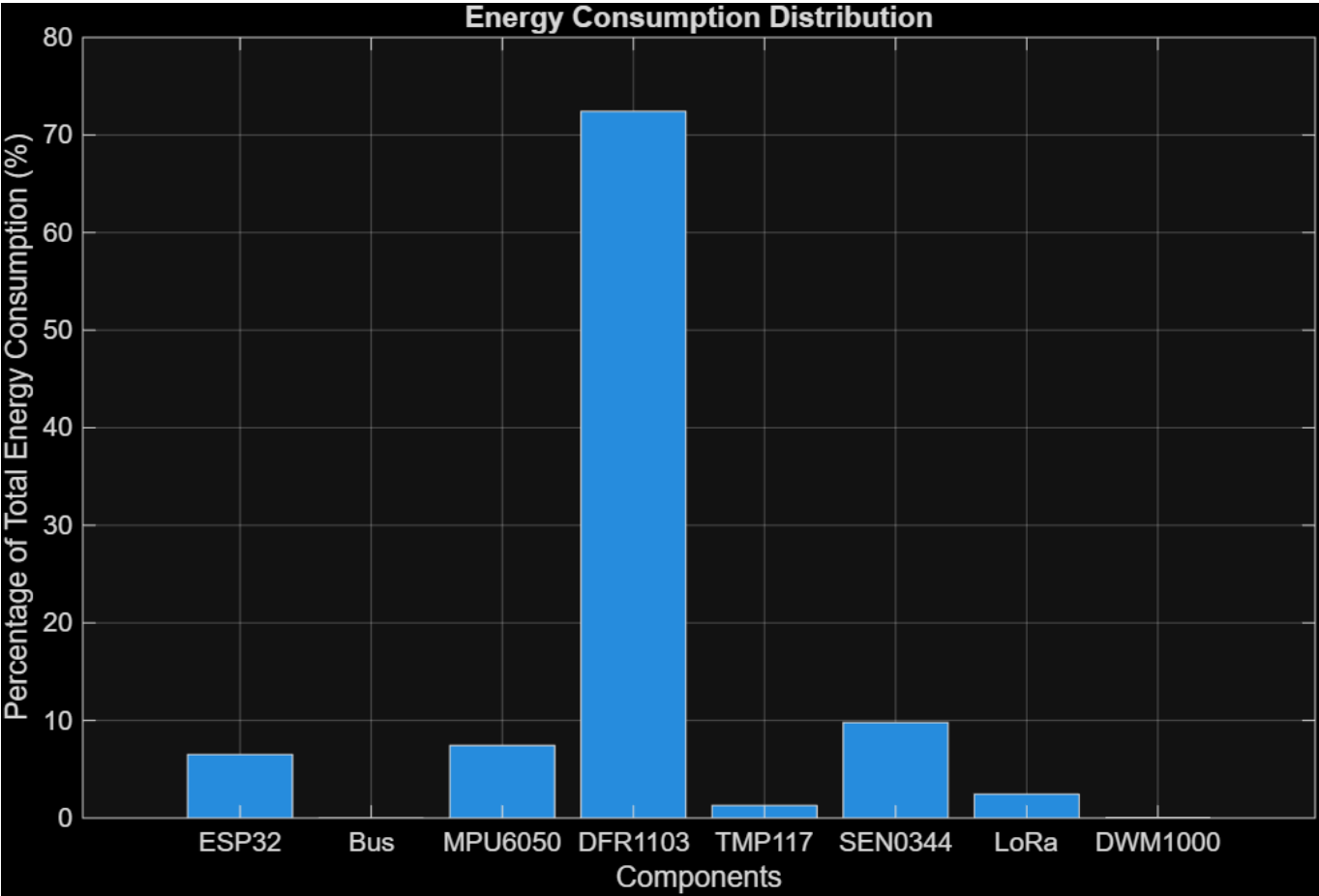
$I_{max} = 425.46 \text{ mA}$;

Para **8h** de duração:

Bateria	Real	c/ Margem de Erro
1 Cell	614 mAh	920 mAh
2 Cell (series)	307 mAh	460 mAh

Regulador LOD tem que aguentar picos de corrente maiores que 640 mA , isto é, $> I_{max} * 1.5$.

Na figura seguinte, conseguimos concluir que o maior consumo de corrente é devido ao módulo de GNSS, seguido das comunicações por wireless LoRa (e futuramente o UWB).



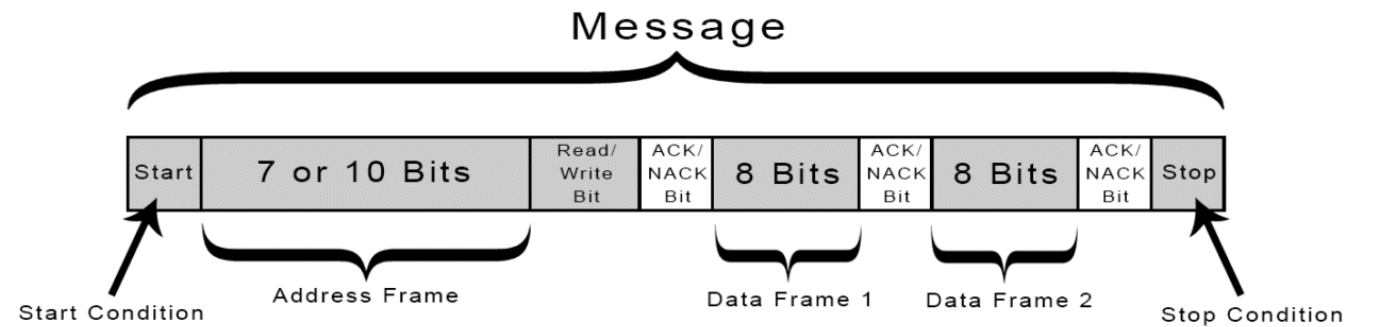
Code Metodology

Para a metodologia do código, só estão as partes relevantes que necessitam de ser explicadas. O código completo, com as definições utilizadas e com os cálculos (ainda que não totalmente corretos) do UWB estará disponível para consultar, mas não vai ser abordado neste documento.

Sensors Enerergy Consumption

O cálculo da energia relativo a cada sensor começa por determinar o tempo de transmissão de dados nos barramentos de dados utilizados.

- Para o barramento I2C temos uma utilização 20 bits de Overhead (caracterizado pelo Start bit, Address Frame de 7 bits, R/W bit, Stop bit e o primeiro byte de dados que corresponde ao registo de onde a ESP vai ler ou escrever dados).



- Já para a transmissão por UART, não existem grandes overheads caracterizado por não existir endereço do destinatário, visto que a comunicação é ponto a ponto. Assim a transmissão por UART é apenas

caracterizada pela baudrate utilizada e a quantidade de bytes enviado. Os 2 bits extra são o Start e o Stop. Poderia ter sido adicionado bits de paridade (entre 0 a 2), mas não vai fazer grande alteração nas contas.

```
for k = 1:length(sensors)

    % Necessary Time for the transmission of all Data of the Sensors
    if strcmp(sensors(k).bus, 'I2C')
        T_comms_sensor(k) = ((sensors(k).bytes * (8+1)) + 20) / bus.I2C_freq;
    elseif strcmp(sensors(k).bus, 'UART')
        T_comms_sensor(k) = (sensors(k).bytes * (8+2)) / bus.UART_baudrate;
    end

    % ESP32's Time Occupation Rate (Inside the T_cicle: Now configured as 10s)
    T_esp_awake_sensor(k) = T_comms_sensor(1, k) * ((T_cycle * sensors(k).fs) /
sensors(k).FIFO_samples);

    % Energy Used by Sensor
    if (sensors(k).always_active)
        Q_sensor(k) = sensors(k).I_active * T_cycle;
    else
        T_active_sensor = (sensors(k).T_wake + sensors(k).T_conversion +
T_comms_sensor(k)) * (T_cycle * sensors(k).fs);
        Q_sensor(k) = (sensors(k).I_active * T_active_sensor) +
(sensors(k).I_sleep * (T_cycle - T_active_sensor));
    end
end

Q_sensor(3) = Q_sensor(3) * 3;
T_esp_awake_sensor(3) = T_esp_awake_sensor(3) * 3;
```

- Como são 3 sensores de Temperatura, a energia foi multiplicada por 3, assim como o tempo em que a esp está acordada devido aos sensores.

Bus Energy Consumption

O único barramento que drena corrente é o barramento do I2C devido a uma resistência de pullup em ambas as linhas (SDA e SCL). Uma vez que a existência de um **bit 0** ou um **bit 1** é equiprovável, * 0.5 é a probabilidade de um bit 1 no barramento (o que vai originar corrente na resistência de pullup).

```
I_I2C_pullup = bus.I2C_Vcc/bus.I2C_R_pullup * 0.5 * 2;
Q_bus_I2C = I_I2C_pullup * sum(T_esp_awake_sensor);
Q_bus = Q_bus_I2C; % Porque Q_SPI = 0 e Q_UART = 0 (não têm R_pullup)
```

LoRa Energy Consumption

No código de matlab estão considerados 3 casos: **Vest Connected (Tx + Rx Windows)**, **Vest Connected (Tx + Rx Windows) + Retrys**, **Vest Disconnected**. O snippet abaixo só demonstra o 2 caso que é o que

apresenta um maior consumo de energia: **Vest Connected (Tx + Rx Windows) + Retrys.**

Notas importantes:

- Os valores do vetor **ToA_tested** foram testados experimentalmente (presente na documentação do LoRa).
- O **Retry_Factor** determina que para cada mensagem que o coleto têm de enviar, é necessário uma média de 1.75 pacotes. Este foi um valor escolhido de forma arbitrária, contudo é bastante pessimista que determina que o sistema envia quase sempre 2 mensagens, aumentando mais a energia gasta teórica, aproximando-se mais do real (e consequentemente, a bateria durar as 8h que foi decidido).
- Para ser possível haver Retrys, após o envio da primeira mensagem, o LoRa tem que abrir, por um determinado tempo (no código é demonstrado pela variável **module_lora.T_rx_window**). Como o código está feito, a janela para Retry só é aberta a partir da primeira mensagem e quando acabar o tempo, já não abre mais (mesmo que haja tentativas de contacto). Se for necessário mudar este algoritmo é preciso modificar esta parte do código.

```
ToA_tested = [51.5e-3, 100e-3, 170e-3, 330e-3, 575e-3, 1.2];
ToA_lora_packet = ToA_tested(module_lora.SF - 6);
Retry_Fator = 1.75; % Sending the same message 1 or 2 times
Num_tx_lora_real = (T_cycle / module_lora.Interval) * Retry_Fator;

Q_lora = (module_lora.I_active_tx * ToA_lora_packet * Num_tx_lora_real) + ...
          (module_lora.I_active_rx * module_lora.T_rx_window *
Num_tx_lora_real) + ...
          (module_lora.I_sleep * (T_cycle - (ToA_lora_packet +
module_lora.T_rx_window)*Num_tx_lora_real));
```

ESP32 Energy Consumption

Para os cálculos da energia gasta pela ESP32, foi definido 50ms para o total que a ESP leva a processar o código de qualquer algoritmo. Este valor é perfeitamente suficiente para as nossas aplicações.

Nota: Esta estimativa é feita para algoritmos leves. Algoritmos muito complexos ou por exemplo filtros de kalman fazem a ESP trabalhar muito mais, pelo que 50ms não seria necessário

```
T_processing = 0.050; % Tempo que a ESP leva a correr o código

Tempo_active_comms = (ToA_lora_packet * Num_tx_lora_real) + T_uwb_active_total;
T_esp_active_total = sum(T_esp_awake_sensor) + Tempo_active_comms + T_processing;
T_esp_sleep_total = T_cycle - T_esp_active_total;

Q_ESP = (ESP.I_active * T_esp_active_total) + (ESP.I_light_sleep *
T_esp_sleep_total);
```

Battery Capacity

A energia consumida pelo Colete é a soma de todas as energias consumidas calculadas anteriormente. A bateria teórica é dada por **Battery_Capacity_Necessary**, no entanto como as baterias reais não podem

chegar aos valores máximos nem mínimos, é boa prática multiplicar por um fator de ***1.5**.

Como existem aproximações nos cálculos (mesmo que o código seja alterado), e na altura em que formos escolher uma bateria os algoritmos possam ainda não ser definitivos **Bateria_com_Fator_Cagaco** dá uma folga (com um fator de ***1.5**) para o valor real da bateria.

```
Q_total_vest = Q_ESP + sum(Q_sensor) + Q_bus + Q_lora + Q_uwb;
I_average = Q_total_vest / T_cycle;
Battery_Capacity_Necessary = I_average * 8 * 10^3;
Bateria_com_Fator_Cagaco = Battery_Capacity_Necessary * 1.5 * 1.5;
```

Peak Current

Num sistema alimentado a baterias, é necessário existir um **regulador LDO** (Low-Dropout Regulator) que assegura uma queda de tensão constante, mesmo quando a ESP32 pede mais do que precisa. No entanto, os LDOs são projetados para aguentar uma dada corrente máxima. Assim, é necessário calcular a corrente máxima que o sistema pode gastar num dado instante.

A **I_Max_total** é estimada a partir da soma de todas as correntes máximas de cada módulo, isto é, como se estivesse tudo a trabalhar ao mesmo tempo.

```
I_peak_sensors = 0;
for k = 1:length(sensors)
    if strcmp(sensors(k).Name, 'TMP117')
        I_peak_sensors = I_peak_sensors + (sensors(k).I_active * 3); % 3x TMP
    else
        I_peak_sensors = I_peak_sensors + sensors(k).I_active;
    end
end

I_peak_LoRa = module_lora.I_active_tx;
I_peak_UWB = module_uwb.I_active_tx;
I_peak_ESP = 100e-3;

I_Max_total = I_peak_sensors + I_peak_LoRa + I_peak_UWB + I_peak_ESP;
```